CSA04-Operating Systems

TASK 6 – Evaluate the Cost of System Calls

Name: V. Reddy Sreya **Regno: 192525124**

1. Evaluation of System Call Costs

System calls allow user programs to access operating system services such as file reading, writing and memory management. The number of system calls directly affects program performance.

1. Why does each system call cost more than an ordinary function call?

A system call requires switching from user mode to kernel mode. It involves parameter validation, security checks and context-switching overhead. An ordinary function call executes within the same process and does not require switching to kernel mode. Therefore, system calls take more time.

2. How does buffer size affect read() and write() calls?

Buffer size determines how much data is transferred in one system call. A smaller buffer requires more read() and write() calls, increasing overhead. A larger buffer transfers more data per call and reduces the total number of system calls.

3. How does library buffering reduce system calls?

Library functions such as fread() and fwrite() use internal buffers to collect data before accessing the operating system. Instead of making a system call for every small read or write, they transfer larger blocks of data. This reduces system-call overhead and improves performance.

4. Best buffer size for the hospital backup program

Based on the given measurements, a 4096-byte buffer is better than 1-byte and 512-byte buffers because it requires only 4 read() calls for the 10,000-byte file. For a real hospital backup program, larger buffers such as 8192 bytes or more can also be tested to identify the best performance.

2. Complete the Code

The following C program copies a file using read() and write() system calls.

Completed TODO 1:

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];

    if (bufsize == 0 || bufsize > sizeof(buf))
        return -1;

    int in = open(src, O_RDONLY);
    if (in < 0) return -1;

    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
        close(in);
        return -1;
    }

    long calls = 0;
    ssize_t n;

    // TODO 1: Read and write loop
    while (1) {
        n = read(in, buf, bufsize);
        calls++;

        if (n == 0)
            break;

        if (n < 0) {
            if (errno == EINTR)
                continue;
            calls = -1;
            break;
        }

        ssize_t total = 0;

        while (total < n) {
```

```
            ssize_t w = write(out, buf + total, n - total);

            if (w < 0) {
                if (errno == EINTR)
                    continue;
                calls = -1;
                break;
            }

            if (w == 0) {
                calls = -1;
                break;
            }

            total += w;
        }

        if (calls == -1)
            break;
    }

    close(in);
    close(out);

    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");

    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);

    fclose(f);

    size_t sizes[] = {1, 512, 4096};

    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n",
               sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));

    unlink("records.dat");
```

```
    unlink("copy.dat");

    return 0;
}
```

3. Expected Output

Expected result
buffer   1 bytes: 10001 read() calls
buffer 512 bytes: 21 read() calls
buffer 4096 bytes: 4 read() calls

Explanation:

- 1 byte: 10,000 data reads + 1 EOF read = 10,001 calls.
- 512 bytes: 20 data reads + 1 EOF read = 21 calls.
- 4096 bytes: 3 data reads + 1 EOF read = 4 calls.

A larger buffer significantly reduces the number of read() system calls.

4. Execution in Linux

Follow these commands in the Linux terminal.

Step 1: Create the C file

nano copy.c

Paste the completed program and save it.

Step 2: Compile the program

gcc copy.c -o copy

Step 3: Execute the program

./copy

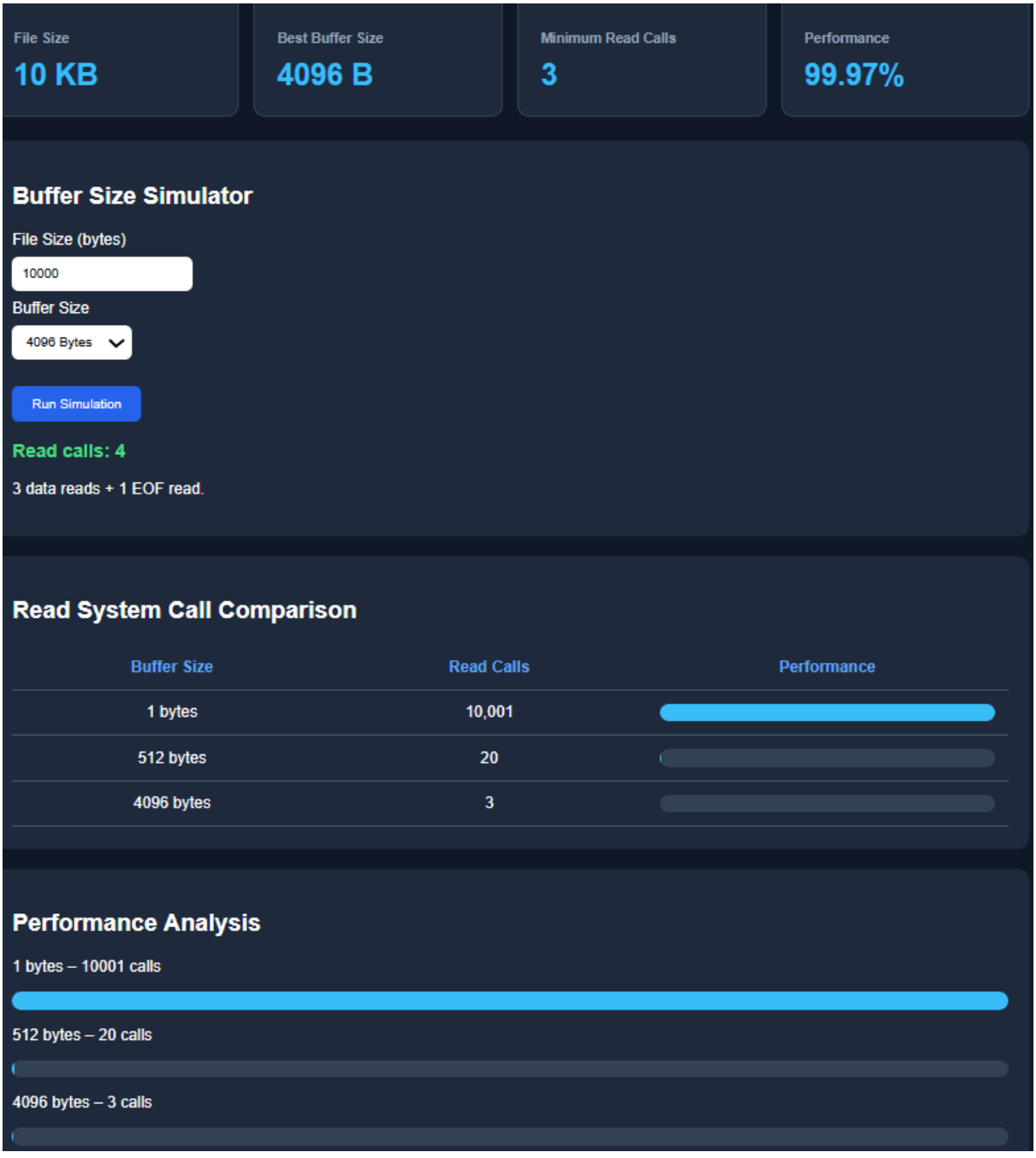
Step 4: Run using strace

strace -c -e trace=read,write ./copy

The strace command displays the number of read() and write() system calls and their execution time.

Note: The program's counter measures only read() calls inside copy_file(). The strace summary measures system calls made by the entire process, so the counts may differ slightly.

OUTPUT:



5. Alternative Concept – Memory-Mapped Files (mmap)

Memory-mapped files use the mmap() system call to map a file directly into a process's virtual address space.

Instead of repeatedly calling read() and write(), the program accesses file contents through memory.

Advantages:

1. Reduces repeated read() and write() system calls.
2. Provides convenient memory-based file access.
3. Can reduce explicit copying between kernel and user space.
4. Useful for accessing large files.

Disadvantages:

1. Requires virtual address space for the mapped file.
2. Page faults may occur when accessing unmapped pages.
3. Very large files require careful memory mapping.
4. Writing changes back to storage may require synchronization.

Comparison:

Method	System calls	Performance
Small buffer	Many	Slower
Large buffer	Fewer	Faster
mmap()	Few mapping calls	Can be efficient

For copying a file with mmap(), both source and destination can be mapped, and data can be copied between the mapped regions. This avoids repeated explicit read() and write() calls, although mapping, synchronization and page-fault operations still have costs.

6. Measured Results and Recommendation

Buffer size	read() calls
1 byte	10,001
512 bytes	21
4096 bytes	4

Number of read() system calls

Comparison for copying a 10,000-byte file.

03K6K9K12K1 byte512 bytes4096 bytes

Recommendation: A 4096-byte buffer is the best choice among the three tested sizes based on system-call count. It reduces the number of read() calls significantly and improves file-copying efficiency. For the hospital backup system, actual execution times should also be measured using larger files and different buffer sizes before selecting the final configuration.

7. Result

The program was designed to copy files using read() and write() system calls with different buffer sizes. The results demonstrate that increasing buffer size reduces the number of system calls and their overhead. Therefore, using an appropriate buffer size improves operating system performance and makes hospital data backups more efficient.